

Start/End Stub 추출 알고리즘 — 상세 기술문서

Routing3D · 기존설계 배관 양 끝 스텝(수직+엘보) 추출 파이프라인 · 단위 mm · 코드 1:1
대응(pattern_learn.py / StubExtractor.cs)

1. 개요 — 스텝 추출이란

Routing3D 자동라우팅은 한 배관 경로를 '양 끝의 짧은 진출/진입 구간(스텝)'과 '가운데 자유공간 구간'으로 나눠 다룬다. 사람이 그린 기존 설계배관(TB_ROUTE_PATH)에서 이 양 끝 스텝의 형상을 학습하려면, 먼저 폴리라인에서 '어디까지가 스텝인가'를 결정론적으로 잘라내야 한다. 본 문서는 그 잘라내기(추출) 알고리즘만을 단계별로 상세히 기술한다.

- 출발 스텝(Start Stub, anchor_kind=EQUIP): 출발 PoC(메인 장비) 쪽 끝. 장비 면에서 나와 수직으로 오른 뒤 첫 엘보(수직→수평 전환)로 주배관랙/자유공간에 올라타는 국소 형상.
- 종단 스텝(End Stub, anchor_kind=DUCT): 종단 PoC(덕트·레터럴) 쪽 끝. 자유공간에서 접근해 수직(주로 상부 +z)으로 덕트에 진입하기 직전까지의, 진입면에서 거꾸로 본 국소 형상.

핵심 설계 결정: 스텝은 '수직배관까지'가 아니라 '수직배관 + 첫 엘보(+짧은 수평 리드인)'까지로 본다. 엘보 방향이 곧 '배관이 어느 쪽 랙으로 빠지는가'를 결정하므로, 엘보를 포함해야 학습된 스텝이 자동설계에서 의미 있는 방향 정보를 갖는다. 전체 수평 랙런은 스텝이 아니다(가운데 자유 구간 소관).

동일 추출 로직이 두 곳에서 1:1 로 쓰인다 — ① 오프라인 학습(Python pattern_learn.walk_stub) 이 특징벡터를 만들고, ② 온라인 뷰어(C# StubExtractor) 가 같은 컷으로 스텝을 표시·고정 라우팅한다. 그래서 화면에 보이는 빨강(출발)·파랑(종단) 스텝이 학습 데이터의 스텝과 정확히 같은 구간이다.

단위는 모두 mm, 좌표는 BIM 과 동일한 월드 프레임. 본 문서의 모든 수치·분기는 구현 코드와 1:1 대응한다(코드 출처는 표지 주석).

2. 입력과 출력

2.1 입력

항목	설명	출처
폴리라인 points	기존배관 중심선 정점열 [(x,y,z), ...] (mm)	ExistingPipe.points / .Points
source_pos	출발 PoC 좌표(장비 측). 폴리라인 정렬(front) 기준	TB_ROUTE_PATH.SOURCE_POS
target_pos	종단 PoC 좌표(덕트·레터럴 측)	TB_ROUTE_PATH.TARGET_POS
앵커 AABB	PoC 를 품는/최근접 장비·덕트 박스(정규화용, 학습 전용)	TB_BIM_EQUIPMENT / TB_DUCT_LATERAL

2.2 출력

- 추출 핵심(공통): 스텝 점열 stub_pts = [PoC, ..., 컷 지점] (월드 mm) + 방향 시퀀스 dir_seq (엘보 포함, 예 [-z,+x]).
- 학습 경로(Python)만: 위 점열을 정규화해 face-rise-offset-rel_pos·feat(24D)·dir_unit(3D) → StubSampleRow → pgvector route_stub_pattern 적재.
- 표시·라우팅 경로(C#)만: 점열을 그대로 빨강/파랑 튜브로 그리고(관경=배관 외경), 스텝 끝점을 A* 시작/목표로 고정(스텝 라우팅).

3. 좌표 규약 — 6 직교 축 스냅

배관은 직교(맨해튼) 형상이지만 BIM 정점은 미세 사선·웁셋이 섞여 있다. 추출의 모든 방향 판정은 임의 3-벡터를 6 직교 축 인덱스(0..5)로 스냅해 이뤄진다 — '가장 큰 절대성분 축'의 부호를 택한다.

인덱스	축	단위벡터
0	+x	(+1, 0, 0)
1	-x	(-1, 0, 0)
2	+y	(0, +1, 0)
3	-y	(0, -1, 0)
4	+z	(0, 0, +1)
5	-z	(0, 0, -1)

축(axis) = 인덱스 // 2 (0=x, 1=y, 2=z), 부호 = 인덱스 % 2 (0=+, 1=-). '수직축'은 첫 방향
 런의 축으로 정의한다(대개 z, 그러나 장비가 옆면으로 PoC 를 내면 x/y 일 수도 있다 —
 코드는 z 를 가정하지 않고 첫 런 축을 동적으로 쓴다).

```
# pattern_learn.axis_snap / StubExtractor.AxisSnap - 동일
def axis_snap(d):
    ax = argmax(|dx|, |dy|, |dz|)
    return ax*2 + (0 if d[ax] >= 0 else 1)
```

4. 추출 파이프라인 개요

한 폴리라인 끝에서 스텝을 잘라내는 흐름은 다섯 단계다. ①②③④ 는 _walk_stub 내부, ⑤
 정렬은 ForPipe/learn_pipe 에서 양 끝을 각각 front 로 두기 위해 선행한다.

- ⑤ 정렬 폴리라인을 'PoC 가 맨 앞(seg[0])'이 되도록 방향 맞춤 (출발/종단 각각)
 - ① 방향 런 압축 세그먼트를 6 축 스냅 → 연속 동일 방향 병합 → [[축 d, 누적길이], ...]
 - 예) 9 정점 → runs = [[-z,1800],[-y,120],[-z,300],[+x,2600]]
 - ② 지터 흡수 250mm 미만 런을 인접 런에 흡수(설계 지터 제거)
 - 예) [-y,120] 흡수 → [[-z,2220],[+x,2600]]
 - ③ 엘보 탐지·길이 첫 런 축=수직축. 축이 다른 첫 런=엘보. 길이는수직누적 + min(엘보런, 800)
 - 예) vert=z, elbow=1(+x) → length = 2220 + min(2600,800) = 3020
 - ④ 점열 절단 seg[0]부터 length 까지 점열 절단(마지막 세그먼트는 보간해 끝점)
 - 예) stub_pts, dir_seq=[-z,+x]

단계	Python (pattern_learn)	C# (StubExtractor)
⑤ 정렬	_oriented / learn_pipe	ForPipe (SourcePos 기준 Reverse)
① 런 압축	_dir_runs	DirRuns
② 지터 흡수	_merge_short_runs	MergeShort
③ 엘보·길이	_walk_stub (본체)	WalkStub (본체)
④ 점열 절단	_points_until	PointsUntil

5. 단계 ⑤ — 방향 정렬

폴리라인은 DB 저장 순서가 source→target 이라는 보장이 없다. 출발 스텝은 source_pos 가,
 종단 스텝은 target_pos 가 각각 seg[0](PoC)이 되도록 정렬해야 한다. 두 끝점 중 기준 PoC
 에 가까운 쪽을 앞으로 둔다(필요 시 reverse).

```
# C# StubExtractor.ForPipe - 출발/종단 두 스텝을 한 번에
ordered = pts;
if (SourcePos != null && dist(pts[0],SourcePos) > dist(pts[^1],SourcePos))
```

```

ordered.Reverse(); // source 가 front 가 되도록
startStub = WalkStub(ordered); // 출발(EQUIP) 쪽
endStub = WalkStub(reverse(ordered)); // 종단(DUCT) 쪽 = 뒤집어서 또 한 번

```

학습(Python learn_pipe) 은 더 엄밀히, source/target PoC 각각의 최근접 정점 인덱스(i_{src} , i_{tgt})를 찾아 $i_{tgt} > i_{src}$ 인 정상 배관만 채택하고, 출발은 $pts[i_{src}:i_{tgt}+1]$ 정방향, 종단은 그 역순을 seg 로 쓴다. 종단 스텝의 진행 단위벡터(dir_unit)는 출발의 부호 반전(-dir_unit)으로 둔다.

6. 단계 ① — 방향 런 압축 (_dir_runs / DirRuns)

인접 정점쌍마다 ㉠ 세그먼트 길이 L 을 재고($1e-6$ 미만은 중복점으로 폐기), ㉡ 방향을 6 축으로 스냅한다. 직전 런과 방향이 같으면 길이를 누적, 다르면 새 런을 연다. 결과는 [축, 누적길이] 런의 리스트다.

```

def _dir_runs(seg):
    runs = []
    for i in 1..len(seg)-1:
        L = dist(seg[i-1], seg[i])
        if L < 1e-6: continue # 중복점 폐기
        d = axis_snap(seg[i] - seg[i-1])
        if runs and runs[-1].d == d: runs[-1].len += L # 같은 방향 누적
        else: runs.append([d, L]) # 새 방향 런
    return runs

```

효과: 자잘한 정점 수에 무관하게 '방향이 바뀐 곳'만 남는다. 꺾임(엘보) 후보는 이 런 경계에서만 생긴다. 다만 미세 지터(짧은 옴셋)도 런 경계를 만들므로 다음 단계에서 걸러야 한다.

7. 단계 ② — 지터 흡수 (_merge_short_runs / MergeShort)

왜 필요한가: BIM 기준배관에는 수직 상승 중간에 100~200mm 짜리 미세 옴셋(예 -y 로 살짝 비켰다 다시 -z)이 섞여 있다. 이를 그대로 두면 '가짜 엘보'가 되어 ③의 엘보 탐지가 진짜 수직→수평 전환 이전에 멈추고, 꺾임 예산(STUB_MAX_BENDS)도 소진한다. 그래서 STUB_MIN_DIR_RUN_MM(250mm) 미만 런을 인접 런에 흡수한다.

알고리즘: '가장 짧은 런'을 찾아 250mm 이상이면 종료, 미만이면 위치에 따라 흡수하고 반복(런 1 개가 남을 때까지). 흡수 규칙은 네 경우다.

짧은 런 위치	흡수 규칙
맨 앞 (idx=0)	바로 뒤 런에 길이 합산 후 제거
맨 뒤 (idx=last)	바로 앞 런에 길이 합산 후 제거
가운데, 양 이웃 방향 동일	앞·짧은·뒤 셋을 하나로 병합(앞 런에 둘 다 합산, 둘 제거)
가운데, 양 이웃 방향 상이	더 긴 이웃 쪽에 길이 흡수 후 제거(방향은 긴 이웃 유지)

```
def _merge_short_runs(runs):
    while len(runs) > 1:
        idx = argmin(run.len)          # 가장 짧은 런
        if runs[idx].len >= 250: break  # 모두 충분히 길면 종료
        if idx == 0: runs[1].len += runs[0].len; del runs[0]
        elif idx == last: runs[-2].len += runs[-1].len; del runs[-1]
        elif runs[idx-1].d == runs[idx+1].d: # 양 이웃 동일 방향 → 셋 병합
            runs[idx-1].len += runs[idx].len + runs[idx+1].len; del runs[idx:idx+2]
        elif runs[idx-1].len >= runs[idx+1].len:
            runs[idx-1].len += runs[idx].len; del runs[idx] # 더 긴 앞쪽에
        else:
            runs[idx+1].len += runs[idx].len; del runs[idx] # 더 긴 뒤쪽에
    return runs
```

핵심 케이스(양 이웃 동일): -z 가 -y(짧음)로 끊겼다 다시 -z 로 이어지면, 셋이 하나의 긴 -z 런으로 복원된다 → 미세 옴셋이 수직 런을 끊어 가짜 엘보를 만드는 문제를 정확히 해소한다.

8. 단계 ③ — 엘보 탐지와 스텝 길이 결정 (_walk_stub 본체)

압축·정제된 런에서 스텝 끝(컷 지점)을 정한다.

- 수직축 = runs[0] 의 축(첫 방향 런). 대개 z 이나 코드는 가정하지 않고 첫 런 축을 쓴다.
- 엘보 = 인덱스 1 이상에서 '축이 수직축과 다른 첫 런'. 즉 수직→수평(또는 다른 축) 전환점.
- 엘보가 있으면: length = (엘보 이전 수직 런들의 누적) + min(엘보 런 길이, STUB_LEADIN_MM=800). 즉 수직 전체 + 엘보 방향으로 최대 800mm 만 — 엘보 '방향'만 담고 랙 런 전체는 제외.
- 엘보가 없으면(끝까지 같은 축): length = 앞쪽 (STUB_MAX_BENDS+1=4)개 런 누적, 단 STUB_MAX_MM(4000) 상한.
- 두 경우 모두 length 는 STUB_MAX_MM(4000mm) 로 클램프. dir_seq 는 보존한 런들의 축 인덱스 리스트.

```
def _walk_stub(seg):
    runs = _merge_short_runs(_dir_runs(seg))
    if not runs: return [seg[0]], []
    vert_axis = runs[0].d // 2
    elbow = first i>=1 with runs[i].d//2 != vert_axis (else None)
    if elbow is None:
        keep = runs[:4] # STUB_MAX_BENDS+1
        length = min(4000, sum(r.len for r in keep))
    else:
        keep = runs[:elbow+1][:4] # 수직(들) + 엘보
        pre = sum(r.len for r in runs[:elbow]) # 엘보 이전 수직 누적
        length = min(4000, pre + min(runs[elbow].len, 800))
    return _points_until(seg, length), [r.d for r in keep]
```

dir_seq 의 첫 토큰 = 진출/진입 방향(수직), 둘째 토큰 = 엘보 방향. 이 둘이 특징벡터의 1 차/2 차 방향 one-hot 으로 인코딩되어, '어느 면으로 나와 어느 쪽으로 꺾이는가'가 학습된다.

9. 단계 ④ — 점열 절단 (_points_until / PointsUntil)

결정된 length 만큼 seg[0]부터 정점을 따라가며 점열을 만든다. 누적 길이가 length 를 넘는 세그먼트는 선형보간(lerp)으로 정확히 length 지점에서 끝점을 만들어 추가한다 — 그래서 스텝 끝은 정점이 아니라 런 경계/지정 길이에 정확히 놓인다.

```
def _points_until(seg, length):
    out = [seg[0]]; total = 0
    for i in 1..len(seg)-1:
        L = dist(seg[i-1], seg[i]); if L < 1e-6: continue
        if total + L >= length:
            t = (length - total) / L # 0..1
            out.append(lerp(seg[i-1], seg[i], clamp(t,0,1))); break
        out.append(seg[i]); total += L
    return out
```

10. 정규화 (학습 전용) — face-rise-offset-rel_pos

추출된 점열을 앵커(장비/덕트) AABB 로컬 프레임으로 정규화해 위치·크기가 달라도 같은 패턴으로 정렬한다. 표시·라우팅(C#)에는 불필요하고, 특징벡터를 만드는 학습(Python _make_sample)에서만 쓴다.

양	정의	코드
face	PoC 가 가장 가까운 앵커 AABB 면(+x..-z). 면거리 최소	nearest_face
rise_mm	면 법선축으로 스텝 점들이 PoC 에서 이동한 최대 거리(상승)	_make_sample (max p[axis]- poc[axis])

	높이)	
offset_mm	PoC 와 그 면 평면 사이 간극(표면 바깥 여유)	_make_sample (face_plane- poc[axis])
rel_pos	앵커 AABB 내 PoC 상대좌표(축별 [0,1], 퇴화축 0.5)	_rel_pos
dir_unit	시작→종단 진행 단위벡터(종단은 부호 반전)	_unit

11. 특징벡터 feat(24D)

정규화 값들을 성분 스케일이 균형 잡힌 24 차원 벡터로 합친다(one-hot/상대좌표/단위벡터라 그룹 내 L2-코사인 검색이 의미를 갖는다). pgvector 의 feat vector(24) 컬럼에 저장된다.

```
feat(24) = [ face          one-hot 6 ] # 진출/진입 면
            + [ dir_seq[0]  one-hot 6 ] # 1 차 방향(수직)
            + [ dir_seq[1]  one-hot 6 ] # 2 차 방향(엘보) ← 엘보 포함의 핵심
            + [ rel_pos      3       ] # 앵커 내 PoC 상대좌표
            + [ dir_unit     3       ] # 시작→종단 진행 단위
```

12. 워크드 예시 (실측 패턴)

project6 학습에서 관찰된 두 대표 패턴. 엘보 포함 개선 전에는 수직축만 잡혀 dir_seq 가 한 토큰뿐이고 지터가 가짜 엘보를 만들었으나, 개선 후 엘보 방향이 dir2 에 정확히 인코딩된다.

12.1 출발 스텝(EQUIP) — 예: Exhaust/ACID

```
원 폴리라인(정렬 후, source=장비 PoC front):
runs(압축) = [[-z,1900],[-y,140],[-z,...],[+x,2600], ...]
지터 흡수   = [[-z,2122],[+x,2600], ...] # -y,140 흡수
vert=z, elbow=1(+x)
length = 2122 + min(2600,800) = 2922 (≤4000)
⇒ face = -z, dir_seq = [-z, +x], rise ≈ 2122, n_bends = 1
개선 전: dir_seq = [-z,-y,-z] (지터가 가짜 꺾임)
```

12.2 종단 스텝(DUCT) — 같은 배관 반대 끝

```
정렬: target(덕트 PoC)을 front 로 역순
runs      = [[+z,...],[-z,...],[+z,...],[-x,...]]
지터 흡수  = [[+z,425],[-x, ...]]
vert=z, elbow=1(-x)
```

⇒ face = +z, dir_seq = [+z, -x], rise ≈ 425, n_bends = 1
 개선 전: dir_seq = [+z, -z, +z] (상하 지터)

도메인 규칙 입증: 405 표본/38 키 집계에서 EQUIP 스텝은 면이 거의 -z(장비 하부에서 PoC 가 아래로 나옴), DUCT 스텝은 +z(덕트 상부 진입)로 강하게 쏠린다 — 추출 알고리즘이 물리적으로 옳은 면을 잡고 있음을 뒷받침한다.

13. 파라미터

상수	값	의미	단계
STUB_MAX_MM / MaxMm	4000 mm	스텝 최대 누적 길이(상한 클램프)	③
STUB_MAX_BENDS / MaxBends	3	엘보 없을 때 보존할 최대 방향 수(꺾임 한도)	③
STUB_MIN_DIR_RUN_MM / MinDirRunMm	250 mm	이보다 짧은 런=설계 지터(흡수)	②
STUB_LEADIN_MM / LeadInMm	800 mm	엘보 이후 담을 수평 리드인 한도	③
ANCHOR_MAX_MM	3000 mm	AABB 밖이어도 매칭 허용하는 앵커 중심 거리(학습)	정규화
(중복점 임계)	1e-6 mm	세그먼트 길이 이하면 같은 점으로 폐기	①④

14. 엣지 케이스와 불변식

- 정점 < 2 개: 스텝 = [seg[0]] 한 점, dir_seq 빈 리스트 → 학습 표본 생성 안 함(make_sample None).
- 전 구간 동일 축(엘보 없음): 앞 4 개 런까지 4000mm 상한으로 절단. dir_seq 는 단일/소수 토큰.
- 지터가 전부(모든 런 250mm 미만): 흡수 반복으로 결국 런 1 개로 수렴 → 수직만, 엘보 없음 처리.
- 수직축이 z 가 아님: 장비가 측면으로 PoC 를 내면 vert_axis = x/y. 코드는 z 를 가정하지 않으므로 일반적으로 동작.
- 결정성: 같은 폴리라인 → 항상 같은 컷(부동소수 보간 포함 결정적). Python·C# 동일 상수·동일 분기라 두 구현 결과가 일치한다(표시 스텝 = 학습 스텝).

- 중복/0 길이 세그먼트: 길이 1e-6 미만은 ①④ 양쪽에서 폐기해 방향 판정·누적을 오염시키지 않는다.

15. Python ↔ C# 1:1 미리

추출 로직은 두 언어에 동일 상수·동일 분기로 포팅돼 있다. 차이는 '추출 다음에 무엇을 하는가'뿐이다.

구분	Python (pattern_learn)	C# (StubExtractor + SceneViewModel)
목적	오프라인 학습(특징벡터 생성)	온라인 표시 + 스텝 라우팅
추출 함수	_dir_runs/_merge_short_runs/_walk_stub/_points_until	DirRuns/MergeShort/WalkStub/PointsUntil (동일)
정렬	learn_pipe (i_src/i_tgt 인덱스)	ForPipe (SourcePos 기준 Reverse)
추출 후	정규화→feat(24D)→pgvector 적재	빨강/파랑 튜브 렌더(관경=배관 외경) + 스텝 끝점을 A* 시작/목표로 고정
상수	STUB_MAX_MM/MAX_BENDS/MIN_DIR_RUN/LEADIN	MaxMm/MaxBends/MinDirRunMm/LeadInMm (동일 값)

스텝 라우팅(SceneViewModel.BuildEngineForRows): 매칭된 기존배관의 출발/종단 스텝을 '고정 설계 구간'으로 깔고, A* 는 스텝 끝(랙 위 자유공간)~끝만 탐색한다. 표시 경로 = [출발 스텝] + [A* 중간] + [reverse(종단 스텝)]. 즉 추출 알고리즘의 출력이 그대로 자동설계 경로의 양 끝이 된다(학습·표시·라우팅 삼자 일치).

16. 요약

- 스텝 추출 = '수직배관 + 첫 엘보(+짧은 리드인 800mm)'를 결정론적으로 잘라내는 5 단계 파이프라인.
- ① 6 축 스냅 런 압축 → ② 250mm 미만 지터 흡수 → ③ 첫 축전환=엘보로 길이 결정 → ④ 보간 절단.
- 지터 흡수가 핵심: 미세 옅셋이 가짜 엘보를 만들어 진짜 수직→수평 전환을 놓치는 문제를 해소.
- 출력 dir_seq 의 [1 차=수직, 2 차=엘보] 가 특징벡터(24D)의 방향 one-hot 으로 학습된다.

- Python(학습) 과 C#(표시·라우팅) 이 동일 상수·분기로 1:1 미러 → 보이는 스텝 = 학습
스텝 = 라우팅 고정 구간.